

Reducing Repeat VM Incidents

Correlating QEMU/KVM, guest kernel logs, and OCI telemetry

The project explained end to end, in simple terms

The project (as written on the resume): Lowered repeat incidents 35% as measured by incident trends by resolving virtio initialization, disk attachment, filesystem mount, interface discovery, kernel panic, and boot stall failures, correlating QEMU/KVM behavior with guest kernel logs and OCI telemetry.

What is inside this document

- Section 1 - The simple summary (the big picture)
- Section 2 - Key words explained in plain English
- Section 3 - Architecture diagram
- Section 4 - The architecture diagram explained, piece by piece
- Section 5 - Every component explained in detail (including all six failure types)
- Section 6 - The complete workflow, step by step
- Section 7 - A worked example of correlation
- Section 8 - How the 35% was measured with incident trends
- Section 9 - Quick reference and a 60-second explanation

Note: The resume line gives the result and the failure types. This document explains how this kind of work is typically done for Linux VMs on QEMU/KVM and Oracle Cloud. Exact tools, host software, and numbers may differ in your real setup. Numbers in the charts are illustrative examples; your real numbers come from your own incident trends.

1. The Big Picture (Simple Summary)

1.1 The one-line version

The same kinds of VM problems kept coming back. I looked at each problem from three sides at once - the physical host running the VM (QEMU/KVM), the VM's own kernel logs, and Oracle Cloud's monitoring data - lined them up on one timeline, found the real root cause, and fixed it permanently. As a result, repeat incidents dropped by 35%.

1.2 Explained with an everyday example

Imagine a building where the lights in one apartment keep going out every few weeks.

- **The quick fix** is to flip the breaker back on. The lights work - until next time. That is a **repeat incident**.
- **To fix it for good**, you need three views at the same moment: the **building's main electrical panel** (the host - QEMU/KVM), the **apartment's own fuse box and appliances** (the guest - kernel logs), and the **power company's meter readings** (the cloud - OCI telemetry).
- **Lining them up by time** shows the truth: "every time the washing machine starts, the apartment's old wiring overheats". Now you can replace the wiring - and the problem never returns.

This project did exactly that for six common VM failure types. Each was traced to its real cause using all three views, fixed at the source (image, kernel, host configuration, or cloud setup), and protected by new tests and alerts so it could not quietly come back.

1.3 The resume line, phrase by phrase

Phrase	What it means in simple words
Lowered repeat incidents 35%	Problems that had happened before came back 35% less often. Example: 80 repeats a year became 52.
as measured by incident trends	Every incident was tagged by type and cause; counting them over time showed the drop.
virtio initialization	The VM's fast virtual devices (disk, network) failed to start correctly.
disk attachment	An extra disk was attached in the cloud, but the VM did not see it.
filesystem mount	The disk was there, but the VM could not open (mount) it, often stopping the boot.
interface discovery	The VM could not find or correctly name its network card, so it had no network.
kernel panic	The VM's operating system core hit a fatal error and stopped.

Phrase	What it means in simple words
boot stall	The VM got stuck during startup without a clear crash.
correlating QEMU/KVM behavior	Looking at what the host's virtualization software was doing at that moment.
with guest kernel logs	...and what the VM's own kernel was reporting...
and OCI telemetry	...and what Oracle Cloud's monitoring and event records showed - all side by side.

1.4 Why this matters

- **Repeat incidents are wasted effort:** the team fixes the same problem again and again.
- **Each view alone is misleading:** the cloud might say "disk attached" while the VM says "no disk". Only together do they explain the gap.
- **Permanent fixes help everyone:** fixing the image or host config protects every future VM, not just one.
- **Trust:** customers notice when the same problem keeps happening.
- **Guardrails keep it fixed:** new tests and alerts catch the problem early if it ever appears again.

2. Key Words in Plain English

Word	Simple meaning	Example
Incident	An unplanned problem affecting a service.	VM will not boot.
Repeat incident	An incident with the same root cause (signature) as an earlier one.	Third fstab failure this quarter.
Problem record	A record for the underlying cause behind one or more incidents; closed only when fixed for good.	PRB-123: iSCSI mounts before login
Root cause	The real reason a problem happened, not just the symptom.	fstab missing _netdev
Signature	A recognizable pattern (log message + context) that identifies a known problem.	"Dependency failed for /data" + iSCSI disk
Host	The physical server running many VMs.	A KVM server in an OCI data center.
Guest	The VM running on the host.	Customer's Oracle Linux VM
QEMU	Software that creates each VM's virtual hardware.	qemu-kvm process
KVM	Linux kernel feature that runs VMs fast on real CPUs.	/dev/kvm
QMP	QEMU Machine Protocol - a way to ask QEMU questions and get its events.	query-status, BLOCK_IO_ERROR
virtio	Standard, fast virtual devices for VMs (disk, network, random numbers).	virtio-blk, virtio-net
Driver probe	The moment a driver finds and sets up a device.	virtio_net probe
Paravirtualized vs iSCSI attach	Two ways OCI connects extra disks: as a virtual device, or over the network (iSCSI).	
Hotplug	Adding a device while the VM is running.	Attaching a volume live
Mount / fstab	Making a disk's files available at a folder; fstab lists what to mount at boot.	/data
UUID	A unique ID for a filesystem, safer than device names like /dev/sdb.	UUID=1b2c...
nofail / _netdev	fstab options: don't stop boot if missing / wait for the network first.	_netdev,nofail

Word	Simple meaning	Example
Interface / NIC / VNIC	A network card; VNIC is OCI's virtual network card.	ens3, secondary VNIC
udev	Linux service that detects devices and gives them names.	udevadm monitor
DHCP / MTU	Automatic IP address / largest packet size.	MTU 9000
Kernel panic / oops	Fatal kernel error / serious kernel bug report.	
Soft lockup	A CPU stuck in the kernel for too long.	"watchdog: BUG: soft lockup"
kdump / vmcore / crash	Saves the kernel's memory when it panics / the saved file / tool to analyze it.	crash vmlinux vmcore
pvpanic	A small virtual device that tells the host "the guest just panicked".	GUEST_PANICKED event
Steal time	Time the VM wanted CPU but the host was busy elsewhere.	st in top
Entropy	Randomness the kernel needs (for keys); low entropy can stall boot on older systems.	virtio-rng
Boot stall	Startup hangs without a crash.	"A start job is running..."
OCI telemetry	Oracle Cloud's data about your resources: metrics, states, events, console history.	Monitoring, Audit
Console history	A saved copy of the VM's serial console text that OCI can capture.	
Correlation	Putting events from different sources side by side by time and ID to find cause and effect.	
5 whys	Asking "why?" repeatedly until you reach the real root cause.	
Guardrail	A test, alert, or default that stops a fixed problem from quietly returning.	CI attach test

3. Architecture Diagram

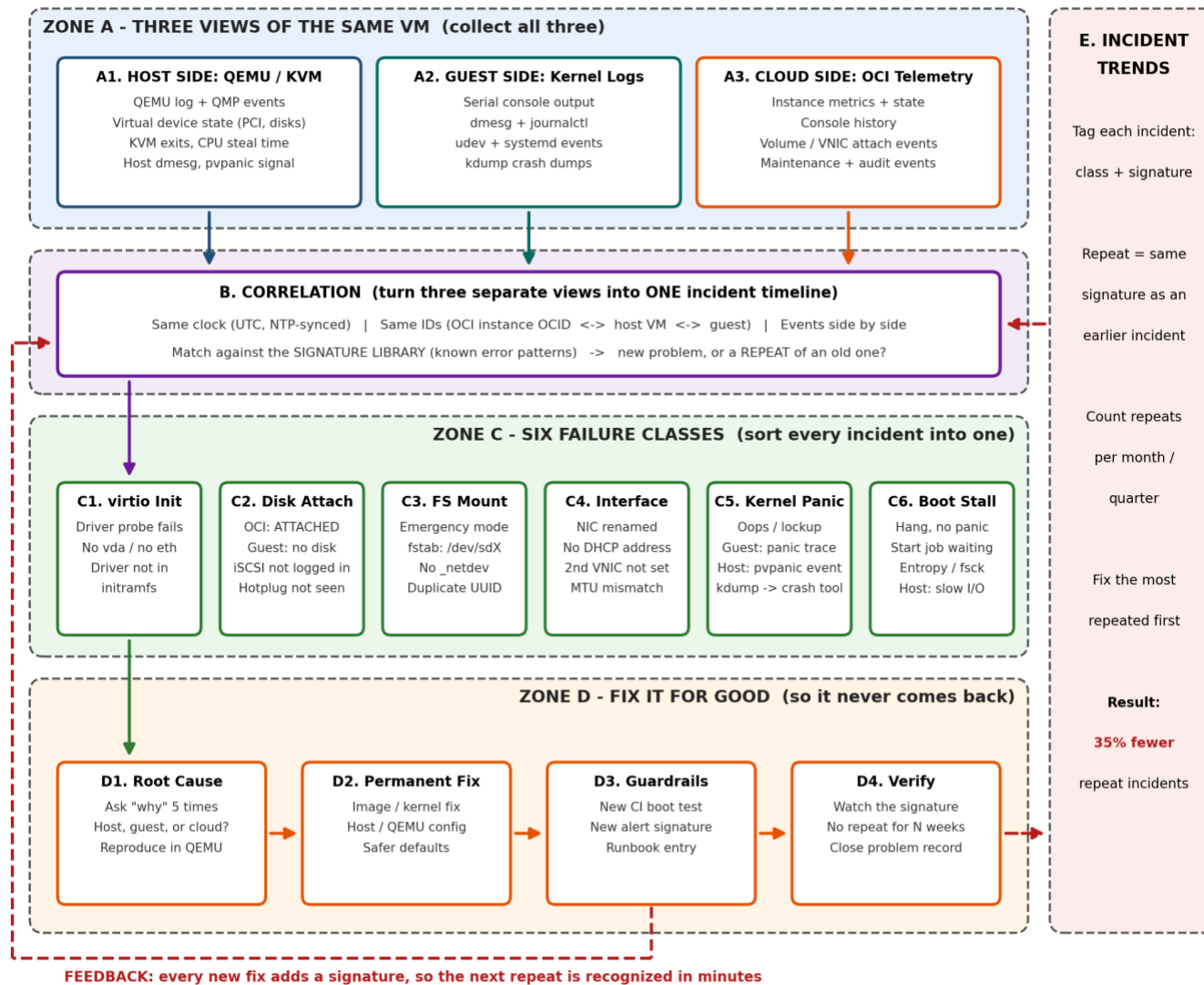


Figure 1 - Three views are correlated, sorted into six failure classes, fixed permanently, and tracked as trends. Labels (A1-E) match Sections 4 and 5.

4. The Architecture Diagram Explained, Piece by Piece

4.1 How to read the diagram

Visual element	What it means
Big dashed boxes (zones A-E)	Groups of parts with the same job: collect, correlate, classify, fix, and measure.
Small solid boxes	Individual components. Labels match Section 5.
Blue / teal / orange boxes in Zone A	The three views: host (blue), guest (teal), cloud (orange). Same colors are used for their arrows.
Purple (Zone B)	The correlation step that joins the three views into one timeline.
Green (Zone C)	The six failure classes. These are categories, not steps, so there are no arrows between them.
Orange (Zone D)	The steps to fix a problem for good.
Red side bar (Zone E)	Incident trends - how repeats are counted and the 35% result.
Solid arrows	The order of work.
Red dashed arrows	Results into trends, priorities back into correlation, and the feedback loop.

4.2 Zone A - Three views of the same VM

- **A1 Host side (QEMU/KVM):** what the virtualization layer saw - device states, QEMU events and errors, CPU steal, and whether the guest signalled a panic.
- **A2 Guest side (kernel logs):** what the VM itself reported - serial console, dmesg, journal, device and service events, crash dumps.
- **A3 Cloud side (OCI telemetry):** what Oracle Cloud recorded - instance state and metrics, console history, disk and network card attach events, maintenance and audit events.
- **Three colored arrows:** all three views flow into correlation. None is enough on its own.

4.3 Zone B - Correlation

One wide box because it combines everything. It puts all events on the **same clock (UTC)**, links them by **the same identity** (the OCI instance ID, the host's VM, and the guest), shows them **side by side**, and compares them with the **signature library** to decide: is this a new problem or a repeat? The **purple arrow** sends the result to classification. The **red dashed arrow** from Zone E tells correlation which repeated signatures to look at first.

4.4 Zone C - Six failure classes

Box	In one line
C1 virtio Init	Virtual disk or network device not set up by the guest driver.
C2 Disk Attach	Cloud says the disk is attached, but the guest cannot see it.
C3 FS Mount	The disk is there, but mounting fails - often stopping the boot in emergency mode.
C4 Interface	Network card missing, renamed, or not configured - no network.
C5 Kernel Panic	The guest's kernel crashes; the host is told through pvpanic; kdump saves evidence.
C6 Boot Stall	Startup hangs without a crash - waiting for something that never comes.

The **green arrow** from C1 down to D1 means: once classified, every incident goes to root-cause analysis.

4.5 Zone D - Fix it for good

- **D1 Root Cause:** ask "why" five times; decide whether the fault is in the host, guest, or cloud side; reproduce it in QEMU.
- **D2 Permanent Fix:** change the image, kernel, host/QEMU configuration, or defaults - not just the one broken VM.
- **D3 Guardrails:** add a CI boot test, a new alert signature, and a runbook entry.
- **D4 Verify:** watch that signature; if there is no repeat for several weeks, close the problem record.
- **Orange arrows:** these steps happen in order.

4.6 Zone E and the feedback loops

- **Zone E Incident Trends:** tag each incident with class and signature, count repeats over time, fix the most repeated first. Result: 35% fewer repeats.
- **Red dashed arrow D4 to E:** verified fixes and their incident data feed the trend.
- **Red dashed arrow E to B:** the trend tells correlation which signatures matter most.
- **Red dashed loop from D3 around the left to B:** every new guardrail adds a signature to the library, so the next repeat is recognized in minutes.

5. Every Component Explained in Detail

The inputs, correlation, fix steps, and trends each have: **In simple terms**, **Detailed breakdown**, **Why it matters**, and an **Everyday analogy**. The six failure classes have a special layout: what each of the **three views** shows, the **common causes**, and the **permanent fixes**.

A1 - Host Side: QEMU/KVM Behavior

In simple terms

What the virtualization software on the physical host saw while the VM was having problems.

Detailed breakdown

- **QEMU log** for the VM: device errors, warnings, crashes of the QEMU process itself.
- **QMP status and events**: is the VM running or paused? Events such as BLOCK_IO_ERROR (disk I/O failed), DEVICE_DELETED (device removed), GUEST_PANICKED (guest crashed, via pvpanic).
- **Device state**: query-pci and query-block show which virtual devices QEMU really gave the VM.
- **KVM behavior**: VM exits and CPU scheduling (perf kvm stat), steal time - is the host too busy?
- **Host dmesg**: host-side storage or network errors that could affect the guest.
- **Maintenance / migration events** on the host at that time.

Example

```
virsh qemu-monitor-command vm1 --pretty '{"execute":"query-status"}'
virsh qemu-monitor-command vm1 --pretty '{"execute":"query-block"}'
perf kvm stat live
```

Why it matters

Separates "the VM broke itself" from "the host did something to the VM".

Everyday analogy: The building's main electrical panel.

A2 - Guest Side: Kernel Logs

In simple terms

What the VM's own operating system reported.

Detailed breakdown

- **Serial console**: every boot message, even when the VM has no network.
- **dmesg / journalctl -k**: driver probes, device detection, I/O errors, oops and panics.
- **journalctl**: service and mount failures, network setup, cloud-init.
- **udev events**: when devices appear and what names they get.
- **Persistent journal**: logs of the failed boot survive a reboot (journalctl -b -1).

- **kdump:** saves a memory dump (vmcore) when the kernel panics, for later analysis.

Example

```
dmesg -T | grep -iE 'virtio|scsi|nvme|i/o error|panic|lockup'
journalctl -b -1 -p err --no-pager
lsblk -o NAME,SIZE,UUID,MOUNTPOINT; ip -br link
```

Why it matters

The guest's point of view shows exactly what it expected and did not get.

Everyday analogy: The apartment's own fuse box.

A3 - Cloud Side: OCI Telemetry

In simple terms

What Oracle Cloud recorded about the VM and its disks and network cards.

Detailed breakdown

- **Instance state and metrics:** RUNNING/STOPPED, CPU, memory, disk and network metrics, health status.
- **Console history:** OCI can capture the serial console text, even when you cannot log in.
- **Attach events:** volume and VNIC attachments with their state (ATTACHING, ATTACHED) and time.
- **Block volume metrics:** throughput and latency for the boot and data volumes.
- **Audit events:** who did what and when (for example AttachVolume).
- **Maintenance events:** planned or unplanned host maintenance affecting the VM.

Example

```
oci compute instance get --instance-id <OCID>
oci compute console-history capture --instance-id <OCID>
oci compute volume-attachment list --instance-id <OCID>
oci compute vnic-attachment list --instance-id <OCID>
```

Why it matters

Shows what the platform believed happened - often the missing piece in the story.

Everyday analogy: The power company's meter readings.

B - Correlation (one timeline)

In simple terms

Joining the three views into a single, time-ordered story of the incident.

Detailed breakdown

- **Same clock:** convert every time stamp to UTC; host, guest, and cloud clocks are all NTP-synced, so events line up.
- **Same identity:** map the OCI instance OCID to the host's VM (QEMU process / domain) and to the guest (hostname, machine-id).

- **Time window:** collect events from a few minutes before the first symptom to the end of the incident.
- **Side-by-side view:** three lanes (cloud, host, guest) on one timeline.
- **Gap spotting:** look for mismatches such as "cloud: ATTACHED" but "guest: no new disk", or "guest: waiting" while "host: idle".
- **Signature library:** known patterns (log message + context + class). A match means a repeat; no match means a new problem.

Why it matters

Most repeat problems were hidden in the gaps between views.

Everyday analogy: Putting three security-camera videos side by side, synced to the same clock.

Signature Library (inside B)

In simple terms

A catalog of known problems and how to recognize them.

Detailed breakdown

- **Each entry has:** a name, class, the log patterns in each view, the root cause, the fix, and a link to the problem record.
- **Written as searchable patterns** so matching can be automated.
- **Example:** "C3-iscsi-netdev: guest shows 'Dependency failed for /data'; disk is iSCSI; fstab lacks _netdev".
- **Grows with every fix** (from D3).

Why it matters

Turns a repeat from an hour-long investigation into a minutes-long recognition.

Everyday analogy: A doctor's book of known symptoms and diagnoses.

C1 - virtio Initialization Failures

In simple terms

The VM's virtual disk or network device exists, but the guest's driver does not set it up, so the VM has no disk or no network.

What each view shows

View	What you see
Host (QEMU/KVM)	query-pci shows the device is present; QEMU config (for example virtio version options) may not match what the guest supports.
Guest (kernel logs)	"virtio_... probe ... failed", no /dev/vda, no network card; drivers missing from initramfs.

View	What you see
Cloud (OCI)	Instance RUNNING but unhealthy; launch mode (paravirtualized vs emulated) or image capabilities do not match the shape.

Common root causes

- virtio drivers missing from initramfs after a kernel update.
- Image launch mode or firmware settings do not match the machine type.
- Older kernel lacks support for a newer virtio feature.
- Interrupt (MSI-X) vector setup problems on some device counts.

Permanent fixes and guardrails

- Build initramfs with virtio drivers included and not host-only.
- Align image capabilities and launch mode with supported shapes.
- Update to a kernel with the needed virtio fixes.
- CI boot test in each launch mode.

Example commands

```
dmesg | grep -i virtio; lspci -nnk | grep -A3 -i virtio
lsinitrd | grep -E 'virtio_(blk|net|scsi|pci)'
```

C2 - Disk Attachment Failures

In simple terms

A block volume is attached in the cloud, but the VM never sees the new disk.

What each view shows

View	What you see
Host (QEMU/KVM)	For paravirtualized attach: was the device hot-added (QMP events, query-block)?
Guest (kernel logs)	No new disk in lsblk; no SCSI/virtio attach messages; for iSCSI, no active session.
Cloud (OCI)	Volume attachment state ATTACHED at a known time; attachment type iSCSI or paravirtualized.

Common root causes

- iSCSI attachment: the guest never ran the login commands, or sessions are not set to log in automatically after reboot.
- Hotplug event missed; guest needs a bus rescan.
- Device names change (sdb vs sdc) so scripts look in the wrong place.

Permanent fixes and guardrails

- Automate iSCSI login and set node.startup=automatic.
- Use consistent device paths (for example /dev/oracleoci/oracleldb or /dev/disk/by-id).

- udev rule or rescan on attach events.
- CI test: attach, detect, reboot, detect again.
- Alert when cloud says ATTACHED but the guest has no matching disk after N minutes.

Example commands

```
iscsiadm -m session
lsblk; ls -l /dev/disk/by-id/
udevadm monitor --kernel --udev
```

C3 - Filesystem Mount Failures

In simple terms

The disk is present, but the VM cannot mount it. If it is in fstab, boot often stops in emergency mode.

What each view shows

View	What you see
Host (QEMU/KVM)	Disk healthy, no I/O errors - proves it is a guest configuration problem.
Guest (kernel logs)	"Dependency failed for /data", "You are in emergency mode", "Filesystem has duplicate UUID", fsck errors.
Cloud (OCI)	Attach event shortly before reboot; volume cloned from another VM.

Common root causes

- fstab uses /dev/sdX names that change between boots.
- iSCSI disks in fstab without `_netdev` (mounted before network login).
- Missing `nofail`, so any missing data disk stops the whole boot.
- Cloned XFS volumes with duplicate UUIDs.

Permanent fixes and guardrails

- Use `UUID=` in fstab with `nofail`, and `_netdev` for iSCSI volumes.
- Add `x-systemd.device-timeout` to fail fast instead of waiting.
- Generate a new XFS UUID on cloned volumes (`xfs_admin -U generate`).
- Image check: `findmnt --verify` in CI.

Example commands

```
UUID=1b2c... /data xfs defaults,_netdev,nofail 0 0
findmnt --verify
journalctl -b -1 -u data.mount
```

C4 - Interface Discovery Failures

In simple terms

The VM cannot find its network card, or finds it under a different name, so networking is not configured.

What each view shows

View	What you see
Host (QEMU/KVM)	Network device and its MAC present; tap/vhost-net backend healthy.
Guest (kernel logs)	Interface renamed (eth0 vs ens3), "no DHCP lease", secondary interface down, wrong MTU.
Cloud (OCI)	VNIC attached with a MAC and IP; secondary VNIC attached at a certain time.

Common root causes

- Network config tied to an interface name that changed (kernel or udev naming change).
- Config tied to an old MAC after cloning an image.
- Secondary VNIC attached in the cloud but never configured in the guest.
- MTU mismatch causing hangs on large transfers.

Permanent fixes and guardrails

- Match network config by MAC or driver from metadata, not by name.
- Remove old MAC-bound configs during image cleanup.
- Automate secondary VNIC setup (for example with OCI utilities).
- Set MTU from DHCP consistently.
- CI test with a secondary NIC.

Example commands

```
ip -br link; ip -br addr
nmcli device status
journalctl -b -u NetworkManager -u cloud-init
```

C5 - Kernel Panic Failures

In simple terms

The guest's kernel hits a fatal error and stops.

What each view shows

View	What you see
Host (QEMU/KVM)	GUEST_PANICKED event (pvpanic), CPU steal spikes, host maintenance or live migration at that moment.
Guest (kernel logs)	Panic message and call trace on serial console; "soft lockup", "hung_task", NULL pointer; vmcore from kdump.
Cloud (OCI)	Instance becomes unhealthy or reboots; maintenance events; console history shows the panic.

Common root causes

- Kernel or driver bug.

- Soft lockups when the host is overloaded (high steal time).
- CPU feature mismatch after migration between different hosts.
- Out-of-memory panic settings.

Permanent fixes and guardrails

- Kernel patch or version pin.
- Host scheduling / capacity fix when steal time is the trigger.
- Consistent CPU model exposed to guests across the fleet.
- kdump enabled in images so every panic leaves evidence.
- Alert on pvpanic events and signature match.

Example commands

```
kdumpctl status
crash /usr/lib/debug/lib/modules/$(uname -r)/vmlinux /var/crash/*/vmcore
grep -E 'panic|soft lockup|Call Trace' console.log
```

C6 - Boot Stall Failures

In simple terms

The VM gets stuck during startup without crashing - it is waiting for something that never arrives.

What each view shows

View	What you see
Host (QEMU/KVM)	VM running but vCPUs idle (guest waiting), or slow storage backend, or high steal.
Guest (kernel logs)	"A start job is running for...", cloud-init waiting for metadata, long fsck, "random: crng init" very late.
Cloud (OCI)	Instance RUNNING but health check failing; boot volume latency spike; console history stops at the same line.

Common root causes

- Waiting for a device that never appears (see C2, C3).
- Low entropy on older kernels blocking early services.
- cloud-init waiting for metadata before the network is ready.
- Long filesystem checks on large volumes.
- Host storage latency or CPU contention.

Permanent fixes and guardrails

- Add virtio-rng for entropy.
- Device timeouts and nofail so boot continues.
- Correct service ordering for network and cloud-init.

- Schedule fsck sensibly.
- Boot-time SLO alert; systemd-analyze in CI.

Example commands

```
systemctl list-jobs
systemd-analyze critical-chain
journalctl -b -o short-monotonic | less
```

D1 - Root Cause

In simple terms

Finding the real reason, not just the symptom.

Detailed breakdown

- **5 whys:** "VM stuck" - why? "waiting for /data" - why? "disk not logged in" - why? "mount ran before iSCSI login" - why? "fstab lacks _netdev" - why? "image template missing it".
- **Which side owns it:** host, guest, or cloud configuration.
- **Reproduce** in local QEMU (same image, same device setup) to prove the cause.
- **Open or update the problem record.**

Why it matters

Only the real cause can be fixed permanently.

Everyday analogy: Finding the overheating wire, not just resetting the breaker.

D2 - Permanent Fix

In simple terms

Changing the source so no future VM hits the problem.

Detailed breakdown

- **Image fixes:** fstab defaults, initramfs drivers, network config, kdump, virtio-rng.
- **Kernel fixes:** patch or pin.
- **Host / QEMU fixes:** device options, CPU model, capacity.
- **Cloud / automation fixes:** attach scripts, VNIC setup, documentation.
- **Rolled out** through the normal staged release process.

Why it matters

Fixing one VM helps one customer; fixing the source helps all.

Everyday analogy: Rewiring the whole building.

D3 - Guardrails

In simple terms

Protection so the fixed problem cannot quietly return.

Detailed breakdown

- **CI test** that reproduces the old failure (for example attach iSCSI, reboot, check mount).
- **Alert** that fires on the signature.
- **Signature** added to the library.
- **Runbook entry** with the fix steps.

Why it matters

Without guardrails, a later change can bring the problem back.

Everyday analogy: Installing a smoke detector after a small fire.

D4 - Verify

In simple terms

Proving the fix worked in real life.

Detailed breakdown

- **Watch the signature** in incidents and alerts after rollout.
- **No repeat for N weeks** (for example 8) -> close the problem record.
- **If it repeats**, reopen and return to D1.

Why it matters

Closes the loop with evidence.

Everyday analogy: Checking the lights stay on for months.

E - Incident Trends

In simple terms

Counting repeats over time to choose what to fix and prove the result.

Detailed breakdown

- **Tag** every incident with class (C1-C6) and signature.
- **Repeat** = same signature as an earlier incident.
- **Count** repeats per month and quarter, per class.
- **Prioritize** the signatures that repeat most or hurt most.
- **Compare** equal periods before and after; normalize by fleet size if it changed.
- **Result:** 35% fewer repeat incidents.

Why it matters

Shows where effort pays off and proves the improvement.

Everyday analogy: A chart of how often each apartment's lights go out.

6. The Complete Workflow, Step by Step

This follows one incident from the first alert to the trend review.

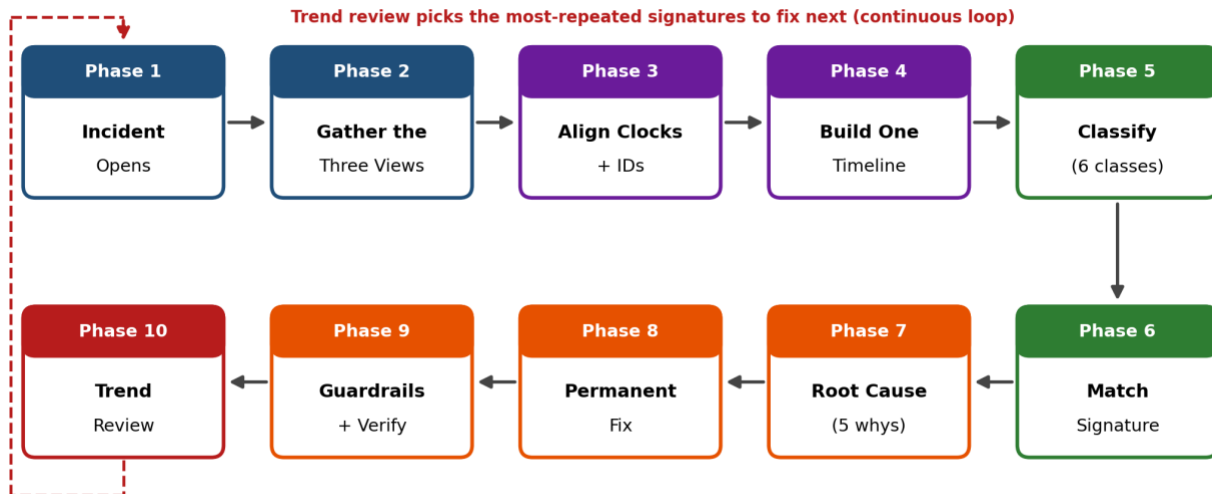


Figure 2 - The 10 phases. Blue = start, purple = correlation, green = classify, orange = fix, red = trends.

Phase 1 - Incident opens

Step	What happens (detail)	In simple words
1.1	Alert, ticket, or failed test opens an incident.	Something is wrong.
1.2	Record instance OCID, time window, image version, shape, and recent changes.	Write down the basics.
1.3	Mitigate if needed (reboot, roll back, detach/reattach) - but save evidence first.	Help the customer, keep the clues.

Phase 2 - Gather the three views

Step	What happens (detail)	In simple words
2.1	Cloud: instance state, metrics, console history, attach events, audit and maintenance events.	Power company readings.
2.2	Host: QEMU log, QMP status/events, device state, steal time, host dmesg.	Main electrical panel.
2.3	Guest: serial console, dmesg, journal (including previous boot), udev, kdump vmcore.	Apartment fuse box.

Phase 3 - Align clocks and IDs

Step	What happens (detail)	In simple words
3.1	Convert all time stamps to UTC.	Same clock.
3.2	Map OCID to host VM to guest identity.	Same apartment.
3.3	Trim to the incident window.	Focus on the right minutes.

Phase 4 - Build one timeline

Step	What happens (detail)	In simple words
4.1	Place events in three lanes: cloud, host, guest.	Three videos side by side.
4.2	Look for gaps and mismatches between lanes.	Where do the stories disagree?
4.3	Mark the first abnormal event.	When did it start?

Phase 5 - Classify

Step	What happens (detail)	In simple words
5.1	Assign one of six classes: virtio init, disk attach, mount, interface, panic, boot stall.	Which kind of problem?
5.2	If more than one, choose the first failure in the chain.	Find the first domino.

Phase 6 - Match the signature

Step	What happens (detail)	In simple words
6.1	Compare with the signature library.	Have we seen this before?
6.2	Match: repeat - link to the existing problem record and apply known fix.	Known illness.
6.3	No match: new problem - open a new problem record.	New illness.

Phase 7 - Root cause

Step	What happens (detail)	In simple words
7.1	Ask why five times until the cause is in a fixable source.	Find the real reason.
7.2	Decide the owner: host, guest image/kernel, or cloud automation.	Whose wiring is it?

Step	What happens (detail)	In simple words
7.3	Reproduce in QEMU to prove it.	Recreate the fault on the bench.

Phase 8 - Permanent fix

Step	What happens (detail)	In simple words
8.1	Change the image, kernel, host config, or automation.	Rewire the building.
8.2	Roll out through staged release.	Carefully, floor by floor.

Phase 9 - Guardrails and verify

Step	What happens (detail)	In simple words
9.1	Add CI test, alert, signature, and runbook entry.	Install the smoke detector.
9.2	Watch for the signature for several weeks.	Check the lights stay on.
9.3	No repeats: close the problem record.	Case closed.

Phase 10 - Trend review

Step	What happens (detail)	In simple words
10.1	Monthly: count repeats by class and signature.	Review the chart.
10.2	Pick the most repeated or most harmful signatures to fix next.	Fix the biggest leak first.
10.3	Compare with earlier periods to measure progress.	Are we improving?

7. A Worked Example of Correlation

A customer reports: "My VM is stuck after a reboot." This had happened many times before; each time someone rebooted it again, and sometimes it worked.

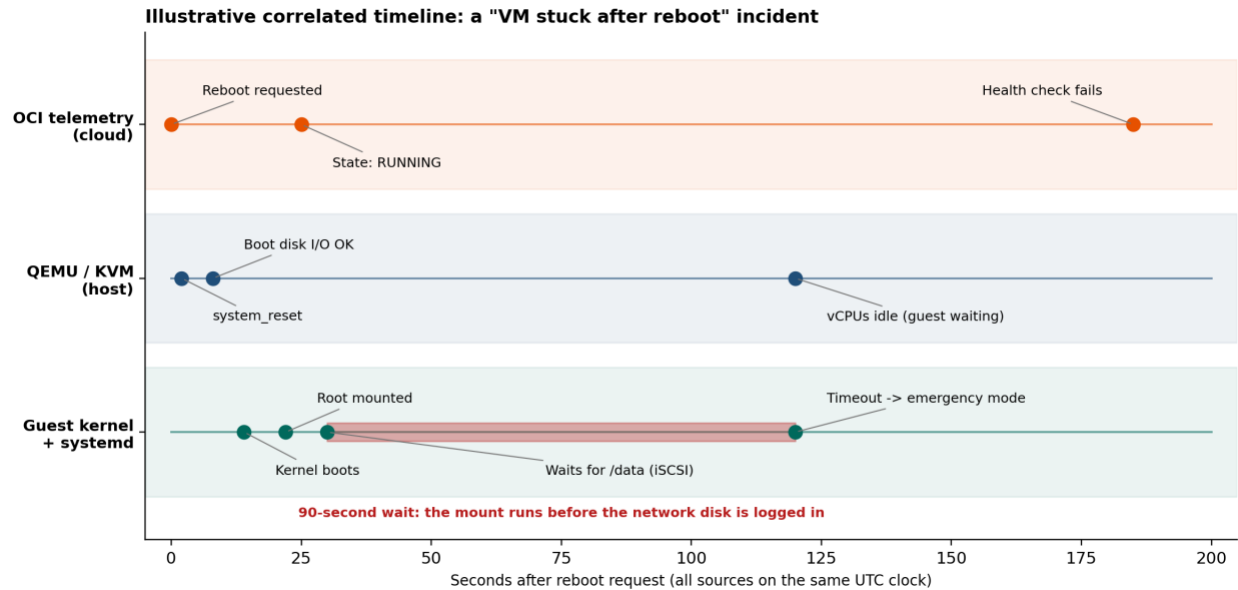


Figure 3 - Illustrative correlated timeline. Each lane is one view; all share the same UTC clock.

View alone	What it seemed to say	Why it was misleading
Cloud only	Instance RUNNING, then health check fails.	Looks like a random failure.
Host only	Boot disk fine, vCPUs idle.	Looks like the VM is healthy but idle.
Guest only	Waiting for /data, then emergency mode.	Looks like a broken disk.

Together: the data disk is an iSCSI volume (cloud attach record), the host saw no disk errors (so the disk is fine), and the guest tried to mount it before the iSCSI login - because fstab did not have `_netdev`. It waited 90 seconds, then dropped to emergency mode.

- **Root cause:** image and documentation templates created fstab entries without `_netdev` and `nofail` for iSCSI volumes.
- **Permanent fix:** corrected the template and attach automation; added the options to existing affected VMs.
- **Guardrails:** CI test (attach iSCSI, add fstab entry, reboot, check boot completes); alert on the signature; runbook entry.
- **Result:** that signature stopped repeating - one of the contributors to the 35% reduction.

8. How the 35% Was Measured with Incident Trends

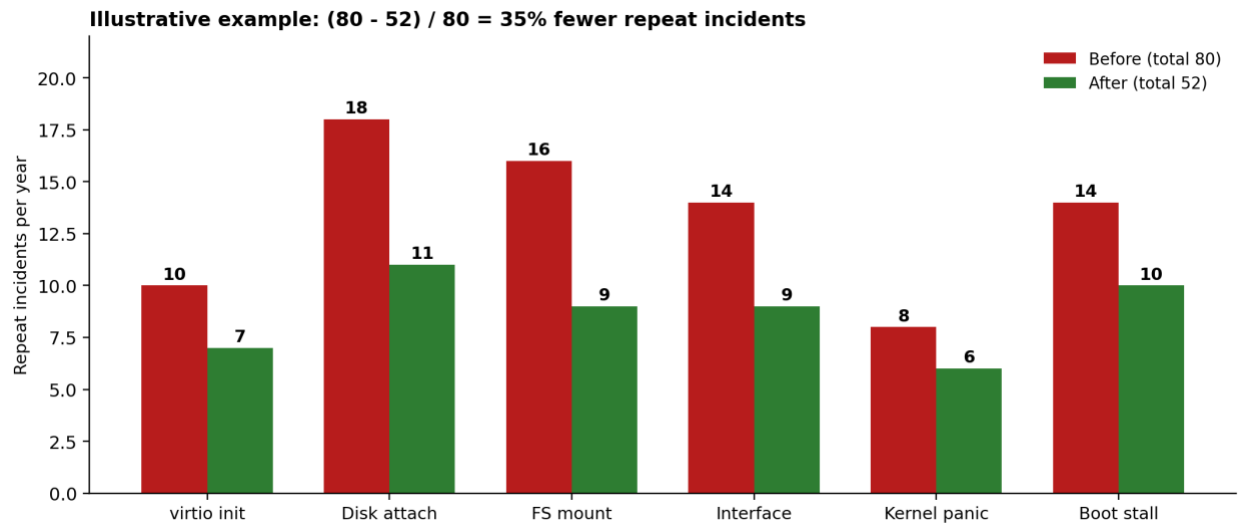


Figure 4 - Illustrative only. Real numbers come from your incident trends.

- **Tag every incident** with its class (C1-C6) and signature.
- **Define a repeat:** an incident whose signature matches an earlier incident.
- **Count repeats** per month and quarter, overall and per class.
- **Compare equal periods** before and after the work (for example 12 months each).
- **Normalize** by fleet size or VM launches if the fleet grew, so growth does not hide improvement.
- **Formula:** reduction = (before - after) / before. Example: $(80 - 52) / 80 = 35\%$.
- **Why not 100%:** some repeats come from customer-side changes or new variants; the remaining ones are the next fix targets.

9. Quick Reference and a 60-Second Explanation

9.1 One-page quick reference

Class	Key signal	Main permanent fix
virtio init	virtio probe failed / no vda or eth	Drivers in initramfs; match launch mode
Disk attach	Cloud ATTACHED, guest no disk	Auto iSCSI login; stable device paths
FS mount	Emergency mode / dependency failed	UUID + nofail + _netdev; unique UUIDs
Interface	Renamed NIC / no DHCP	Config by MAC; automate secondary VNIC
Kernel panic	Panic trace + GUEST_PANICKED	Kernel fix; kdump; consistent CPU model
Boot stall	Start job waiting / vCPUs idle	Timeouts; virtio-rng; service ordering

9.2 The 60-second explanation (for interviews)

"We had VM problems that kept coming back: virtio devices not initializing, attached disks not appearing, mounts failing, network interfaces not discovered, kernel panics, and boot stalls. Each was usually fixed with a reboot, so it returned. I correlated three views on one UTC timeline - QEMU/KVM behavior on the host such as QMP events, device state, pvpanic and steal time; guest kernel logs from the serial console, dmesg, journal and kdump; and OCI telemetry such as instance state, console history, and attach events. The gaps between views revealed root causes, like iSCSI volumes mounted before login because fstab lacked _netdev, or drivers missing from initramfs. I fixed them at the source in images, kernel, host config, and automation, added CI tests, alerts, and signatures, and tracked incidents by signature. Repeat incidents dropped by 35%."